



Discrete Optimization

Exact optimization and decomposition approaches for shelf space allocation

Burcu Caglar Gencosman^{a,*}, Mehmet A. Begen^b^a Department of Industrial Engineering, Bursa Uludag University, Bursa, Turkey^b Ivey Business School, Western University, London, Ontario, Canada

ARTICLE INFO

Article history:

Received 28 July 2020

Accepted 30 August 2021

Available online 5 September 2021

Keywords:

Retailing

Shelf space allocation

Rectangular display problem

Mixed-integer linear programming

Logic-based Benders decomposition

2-stage algorithm

ABSTRACT

Shelf space is one of the scarcest resources, and its effective management to maximize profits has become essential to gain a competitive advantage for retailers. We consider the shelf space allocation problem with additional features (e.g., integer facings, rectangular arrangement restrictions) motivated by literature and our interactions with a local bookstore. We determine optimal number of facings of all products in two aspects (width and height of a rectangular arrangement space for each product), and allocate them as contiguous rectangles to maximize profit. We first develop a mixed-integer linear mathematical programming model (MIP) for our problem and propose a solution method based on logic-based Benders decomposition (LBBD). Next, we construct an exact 2-stage algorithm (IP1/IP2), inspired by LBBD, which can handle larger and real-world size instances. To compare performances of our methods, we generate 100 test instances inspired by real-world applications and benchmarks from the literature. We observe that IP1/IP2 finds optimal solutions for real-world instances efficiently and can increase the local bookstore's profit up to 16.56%. IP1/IP2 can provide optimal solutions for instances with 100 products in minutes and optimally solve up to 250 products (assigned to 8 rows x 160 columns) within a time limit of 1800 s. This exact 2-stage IP1/IP2 solution approach can be effective in solving similar problems such as display problem of webpage design, allocation of product families in grocery stores, and flyer advertising.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Retail shelf space management, deciding which products to be displayed and shelf space to be allocated, is one of the most complex issues of retailing, and the success of a retailer depends on its ability to adapt to the changing environment by continually determining which product will be displayed in what amount and when (Hansen, Raut, & Swami, 2010). Intense competition environment, rapid increase in product variety, variability in customer demands, and limited resources of retail shelf areas are some of the factors that make retail shelf space management complex. However, when done effectively, retail shelf space management can be a competitive advantage (Chen & Lin, 2007). The decisions in the literature for retail shelf space management can be grouped as product assortment planning, shelf space allocation, shelf space layout, and determination of shelf space stock quantities (Hübner, 2017; Özcan, 2010). In this study, we consider a shelf space allocation

problem with specific features such as rectangular arrangement restrictions.

Shelf space allocation problems can be classified according to model structures that consider various factors affecting demand, assortment, and allocating decisions such as space elasticity (Curhan, 1972; Hansen & Heinsbroek, 1979), location elasticity (Chandon, Hutchinson, Bradlow, & Young, 2009), and cross-space elasticity (Bianchi-Aguilar, 2015; Corstjens & Doyle, 1981). Space-elasticity factor represents the effect of number of facing units (number of products in front of the shelf) on demand. Researchers show that number of facings has a significant impact on customers' buying behavior and demand - the more customers see, the more they demand - since customers can only see the front row of the shelf (Chen, Chen, & Tung, 2006; Hübner & Kuhn, 2012; Kök, Fisher, & Vaidyanathan, 2015; Nafari & Shahrabi, 2010). Location elasticity represents the effects of products' vertical and horizontal positions on the demand of products. Top and middle shelf positions (eye and hand level) and shelves that are beginning of the corridor are more effective than other locations (Chandon et al., 2009). Cross-elasticity captures how responsive demand for a good is when the position of another good changes. This is positive for

* Corresponding author.

E-mail addresses: burcucaglar@uludag.edu.tr (B. Caglar Gencosman), mbegen@ivey.uwo.ca (M.A. Begen).

substitute goods and negative for complementary goods (Corstjens & Doyle, 1981; Özcan, 2011). In this paper, we assume that demand is only affected by space elasticity and location elasticity, which is common and well justified in the literature (Table 2).

To make shelf allocation decisions, retailers use planograms which are graphical displays of products' arrangement on shelves. Planograms include the exact (vertical and horizontal) location of each product and its facing units (space assignment), and they are efficient tools to adjust product allocation with dynamic demand and scarce shelf space (Coskun, 2012; Frontoni, Marinelli, Rosetti, & Zingaretti, 2017; Geismar, Dawande, Murthi, & Sriskandarajah, 2015; Van Nierop, Fok, & Franses, 2008). However, production of planograms is difficult and time-consuming because it involves solving a multiple knapsack problem except product location, which is NP-Hard (Martello & Toth, 1990; Yang & Chen, 1999). Hübner, Schäfer, & Schaal (2021) state that there are three decisions in generating planograms: 1-*item selection*, 2-*space assignment*, and 3-*item arrangement*. We can achieve *space assignment* by determining the number of horizontal and vertical facings of products, and *item arrangement* by the rectangular placement of products with specific arrangement constraints in minutes. Therefore, for a given set of selected items, we are able to generate optimal solutions for real-world planogram problems.

This paper presents a shelf space allocation problem (SSAP) with specific features, such as integer facings (the number of units visible to customers in front of the shelf), merchandising rules (rectangular arrangement) and demand function with space and location elasticities. To distinguish our problem from classical SSAPs, we indicate our specific SSAP problem as SSAP-RA; a *Shelf Space Allocation Problem with Rectangular Arrangement*. We determine optimal number of facings of all products in two dimensions (vertical and horizontal), and optimally allocate them as rectangular shapes to maximize profit.

Our proposed exact solution approach ensures rectangular arrangement, one of the popular merchandising rules (Bianchi-Aguiar, Hübner, Carravilla, & Oliveira, 2021) and it can be used for many other similar problems, e.g., shelf space allocation in book retailers, display advertising on webpages, and allocation of product families in rectangular shapes which is common in grocery stores. Grocery stores, book retailers, and webpages need to adjust their shelf spaces and displays rapidly for many products depending on the market, demand, and promotions. They need a fast and effective method to solve their shelf allocation problem.

To the best of our knowledge, our paper is the first to provide an exact solution method IP1/IP2 for practical real-size SSAPs-RA. Besides the real-world instances, we provide two other datasets that are randomly generated using parameters from two different sources: real-world instances and benchmarks from the literature. The IP1/IP2 method can find optimal rectangular allocations of 100 products less than two minutes on average, and it can solve real-world instances up to 250 products within time limit (1800 seconds) on a personal computer (Windows 10 (64-Bit) operating system machine, 8 GB RAM and an Intel Core i7-9750H CPU with 2.59 GHz). Our case study in a local bookstore shows that IP1/IP2 optimally achieves rectangular allocation of books in the bookstore efficiently and it does improve the profit up to 16.56%, on average. Furthermore, SSAP-RA has similarities with two-dimensional (2D) display, 2D cutting, and 2D strip packing. Therefore, an effective solution to SSAP-RA can contribute to the literature for other related problems. Since retailing is an indispensable element of the production/service industry, development of fast and successful solution methods for SSAP-RA can positively contribute to retailers' success in different sectors and provide a competitive advantage.

The organization of the paper is as follows. Section 1.1 defines our problem by using a well-established classification frame-

work of shelf space planning problems from the literature (Bianchi-Aguiar et al., 2021). Section 1.2 provides a literature review on related studies. Section 2 presents the details of SSAP-RA and the mixed-integer programming model (MIP). Section 3 introduces our proposed solution approaches. Section 4 explains the datasets we used and Section 5 represents our results. Finally, we give some concluding remarks in Section 6 and provide the proofs in Appendix A and Appendix B.

1.1. General description of SSAP-RA

There are recent review studies of shelf space planning problems in the literature (Bianchi-Aguiar et al., 2021; Coskun, 2012; Hübner & Kuhn, 2012). A more recent one, Bianchi-Aguiar et al. (2021), presented a comprehensive review on shelf space planning problems and developed a classification framework. To classify the related studies in the literature, they introduced a notation system (Table 1) for different components of shelf space planning problems. Using this notation, we aim to define our SSAP-RA formally and show its position in the literature.

The fundamental classification of shelf space planning problems is the *decisions* to be taken, and depending on the decisions we see four problem types: 1) SP: space assignment problems, 2) SAP: space assignment and vertical allocation problems, 3) SLP: space assignment and horizontal location problems, and 4) SALP: space assignment with vertical allocation and horizontal location problems. There are four major *related planning problems* with shelf space planning: 1) A: assortment planning, 2) P: pricing, 3) M: macro-shelf space planning, and 4) R: replenishment planning. *Demand effects* on shelf space planning problems can be various: 1) spe: space elasticity, 2) csp: cross-space elasticity, 3) oos: OOS (out-of-stock) substitution, 4) ver: vertical demand effect, 5) hor: horizontal demand effect, and 6) arr: arrangement effect. *Demand types* can be grouped as: 1) sto: stochastic, 2) det: deterministic, 3) stat: stationary, and 4) seas: seasonal. The significant *constraints* are described as: 1) int: integer facings, 2) avai: product availability, 3) cap: for capacitated shelf space, and 4) merch: merchandising rules.

According to this classification framework, our problem can be classified as SALP-spe-ver-hor-det-int-cap-merch; a space assignment (S) with vertical allocation (A) and horizontal location (L) problem, which has a deterministic (-det) demand function affected by space-elasticity (-spe) and vertical location-elasticity (-ver), and includes integer facings (-int), capacity (-cap) and merchandising (-merch) constraints.

Besides the formal classification, our problem has similar properties with two-dimensional shelf space allocation. There are two requirements for allocating products to two-dimensional shelves (Hübner et al., 2021). The first one is to ensure two-dimensional product arrangements and provide a total customer perspective. The second one is to produce rectangular facing arrangements. Our problem contains both two requirements. Our proposed models determine horizontal and vertical facings in a rectangular shape, which depend on the arrangement of other items, and provide customers a second perspective, similar to tilted shelves. The other similarity is that the allocation options are not restricted by shelf levels and their height, and the products do not necessarily need to be positioned along a dividing line, as in regular shelves (Hübner et al., 2021). In addition, our problem belongs to the class of two-dimensional knapsack problems, detailed in Section 2.

There are many types of shelf space planning problems in the literature (e.g., see Bianchi-Aguiar et al., 2021; Coskun, 2012; Hübner & Kuhn, 2012), we focus on studies similar to our problem in the next section.

Table 1

The notation for shelf space planning problems by Bianchi-Aguilar et al. (2021).

1. Decisions	3. Demand effects	4. Demand types	5. Constraints
S Space assignment	-spe Space elasticity	-sto Stochastic	-int Integer facings
A Vertical allocation	-csp Cross-space elasticity	-det Deterministic	-avai Availability
L Horizontal location	-oos OOS substitution	-stat Stationary	-cap Capacity
2. Related problems	-ver Vertical demand effect	-seas Seasonal	-merch Merchandising rules
A- Assortment	-hor Horizontal demand effect		
P- Pricing	-arr Arrangement effect		
M- Macro-shelf space			
R- Replenishment			

Table 2

Related studies with SSAP-RA.

Study	Dec.	Demand effects			All developed methods	Solution approach	Items
		-spe	-csp	-oos			
Van Nierop et al. (2008)	SAL	X			SA	SA	407
Hwang et al. (2009)	SAL	X	X		NLP, GA	GA	4
Hansen et al. (2010)	SAL	X	X		MILP, heuristics and GA based meta-heuristic	Heuristics and GA based meta-heuristic	100
Murray et al. (2010)	SA	X			MINLP, BONMIN algorithms	Relaxed B-Hyb and B-OA algorithms	100
Russell & Urban (2010)	SAL	X			MIQP, two-phase heuristic	Two-phase heuristic	103
Bai et al. (2013)	SAL	X	X		NLP, SA hyper-heuristic	SA hyper-heuristic	100
Geismar et al. (2015)	SAL	X			MILP, subproblem decomposition model	Subproblem decomposition model	579
Zhao et al. (2016)	SAL	X	X		MILP, SA based hyper-heuristic	SA based hyper-heuristic	100
Rabbani et al. (2018)	SA	X	X		MINLP, GA	GA	7000
Bianchi-Aguilar et al. (2018)	SAL	X			MILP, matheuristics.	Matheuristics	240
Hübner et al. (2021)	SAL	X		X	MINLP, GA	GA	2000
Our Paper	SAL	X			MILP, LBBD, two-stage exact model	Two-stage exact model	250

1.2. Related studies

There is substantial literature on shelf space allocation problems started with empirical studies date back to the 1960s (Cox, 1970; Kotzan & Evanson, 1969). Since then, a great deal of research has been carried out, including mathematical models of shelf space planning problems (Anderson & Amato, 1974; Corstjens & Doyle, 1981; Urban, 1969; Zufryden, 1986). However, as mentioned before, we concentrate only on studies that are similar to our problem, such as SA or SAL problems that consider vertical allocation and/or horizontal location with a rectangular assignment. Table 2 presents the studies related to our problem. The first column shows the reference and the second column gives the major definition of the problem considered in the related study. The third part indicates the factors that affect the demand function: space elasticity (-spe), cross-space elasticity (-csp), and out-of-stock substitution (-oos). Since all studies in Table 2 consider the vertical demand effect (-ver), the corresponding column is not included. The sixth column shows the developed methods in the corresponding study, the seventh column presents the solution approach used in numerical tests for the maximum number of items, and the last column indicates the maximum number of items considered in their numerical tests. The demand types of studies in Table 2 were deterministic except Hübner et al. (2021), which had a stochastic demand. The abbreviations for models are MILP (mixed-integer linear programming), MIQP (mixed-integer quadratic programming), NLP (non-linear programming), MINLP (mixed-integer non-linear programming), GA (genetic algorithm), SA (simulated annealing), and LBBD (logic-based Benders decomposition).

Most of the papers in Table 2 developed various kinds of heuristics, such as meta-heuristics, hyper-heuristics, or matheuristics to solve complex SA or SAL problems in acceptable times besides mathematical programming models. Van Nierop et al. (2008) developed a statistical model to measure the impact of shelf space and shelf placement on sales, and then they proposed a simulated annealing-based heuristic optimization technique to find

the profit-maximizing shelf layouts in a leading supermarket chain. Hwang, Choi, & Lee (2009) considered the shelf space design and item allocation problem with various brands of items displaying on rectangular shelf space. They introduced a non-linear programming problem with continuous and discrete variables aiming profit maximization and also developed two genetic algorithm-based heuristics. They generated 30 test problems with four brands of items and examined the validity of their solution procedures. Hansen et al. (2010) proposed a linear programming model of a non-linear profit function to solve the shelf-space problem optimally. They found out that the vertical location effect was approximately double the size of the horizontal location effect on profit performance. Since the linear programming model was able to solve smaller problems (up to 20 products), they built two heuristic algorithms and a meta-heuristic algorithm. The comparison studies with larger problems (up to 100 products) showed that the genetic algorithm-based meta-heuristic outperformed the other algorithms. Murray, Talukdar, & Gosavi (2010) constructed a mixed-integer non-linear model (MINLP) that optimizes retailer decisions on product prices, number of facings, display orientation, and positioning on the shelf. To solve their MINLP model, they used an open-source software called BONMIN (Basic Open Source Nonlinear Mixed Integer programming), which provides relaxed versions of the original problem based on algorithms, some of which are heuristics (Bonami & Lee, 2007). They solved the instances with a maximum number of items (100 items) by the B-OA algorithm (an outer-approximation-based decomposition) and the B-Hyb algorithm (a hybrid of the outer-approximation and branch-and-cut algorithms) within acceptable times. Russell & Urban (2010) considered a shelf-management problem with individual items categorized as part of a product family and located as a rectangular display. They proposed a mixed-integer quadratic program with a quadratic objective function and a linear constraint set. They included the effect of shelf location (vertical and horizontal) in a continuous manner. Since the mixed-integer quadratic program could not solve the problems with more than 10 items opti-

mally, they developed a two-phase heuristic algorithm, which was able to find a near-optimal solution of a problem with 103 items, 36 product families, and 25 shelves. Bai, Van Woensel, Kendall, & Burke (2013) studied a two-dimensional shelf space allocation problem and developed an integer non-linear programming model. However, their non-linear model failed to solve medium-sized (29 items) instances. Thus, they turned their attention to heuristic approaches and developed a hybridization of a simulated annealing algorithm with a hyper-heuristic learning mechanism and a gradient approach. The hyper-heuristic approach could solve the instances with 100 items. Geismar et al. (2015) examined the allocation of DVDs in a retail shelf-space providing a contiguous rectangular presentation to maximize revenues. The authors proposed a mixed-integer linear model of the two-dimensional rectangular display problem. For large-scale problems, they decomposed the master problem into two subproblems. While the first problem consisted of allocating products to different cabinets, the second problem included the assignment of products in a contiguous rectangle. Their decomposition approach could solve the instances with 500 products within 5% optimality gap. Zhao, Zhou, & Wahab (2016) developed a joint optimization model for shelf space allocation and display location problem considering joint replenishment policy. They proposed a multi-stage simulated annealing-based hyper-heuristic algorithm to solve the optimization problem, which could solve instances of up to 100 items. Rabbani, Salmanzadeh-Meydani, Farshbaf-Geranmayeh, & Fadakar-Gabalou (2018) studied a shelf retail decision-making model and investigated the effect of various factors such as vertical and horizontal location, product cross-space elasticity and the number of items at eye-level. They considered shelves in 3-dimensions; length, width, and the depth of shelves. Besides a mixed-integer non-linear mathematical model, they proposed a genetic algorithm-based meta-heuristic for larger-scale instances (up to 7000 units). Bianchi-Aguiar, Silva, Guimarães, Carravilla, & Oliveira (2018) considered a shelf space allocation problem with merchandising rules: hierarchical product families and display directions. They proposed a mixed-integer programming model of the problem and developed a mathematical programming-based heuristic (matheuristic) algorithm, which was capable of generating high-quality feasible solutions of various instances up to 240 products. Hübner et al. (2021) extended Geismar et al. (2015) to develop a novel stochastic non-linear mathematical model for two-dimensional shelf-space allocation of items to a tilted shelf considering substitution effects of products. Due to the model's complexity, the authors developed a genetic algorithm-based heuristic with a good performance on large size instances up to 2000 items.

Studies mentioned so far have some similarities with our study, such as the same demand function or rectangular display arrangement. Although most of these studies represented their problems with mathematical models, they developed heuristics, meta-heuristics, or matheuristics to solve large size problems, which resulted in near-optimal solutions. We contribute to the literature by providing exact efficient solution methods for large size SSAPs-RA (up to 250 products and allocated 571 facings): a logic-based Benders decomposition algorithm (LBBD1) and an exact two-stage mathematical model (IP1/IP2).

Logic-based Benders decomposition (LBBD) is one of the recent and popular decomposition methods used for integer mathematical programming models. To date, LBBD has been applied to different kinds of combinatorial problems (Barzanji, Naderi, & Begen, 2020; Camargo, Toledo, & Almada-Lobo, 2014; Cambazard, Hladik, Déplanche, Jussien, & Trinquet, 2004; Coban & Hooker, 2010; Fazel-Zarandi & Beck, 2009; Heinz & Beck, 2012; Hooker, 2005; 2007; Naderi, Roshanaei, Begen, Aleman, & Urbach, 2021; Tran & Beck, 2012; Xiaolu, Baocun, Yingwu, & Feng, 2014). Researchers have used LBBD for similar problems to SSAP-RA, such as 2D strip

packing (Côté, Dell'Amico, & Iori, 2014; Delorme, Iori, & Martello, 2017), 2D cutting (Martin, Hokama, Morabito, & Munari, 2020) and stamping scheduling (Caglar Gencosman, Begen, Ozmutlu, & Öztürk Yılmaz, 2016). To the best of our knowledge, this is the first study that provides an LBBD algorithm for solving SSAP-RA.

In their systematic classification of papers in the literature, Bianchi-Aguiar et al. (2021) placed the rectangular display constraints into the field of merchandising rules. The authors found that only 5% of the papers consider merchandising rules. They stated that the merchandising rules need to be investigated further in more detail, and only a few papers treat these rules as constraints so far in the literature. In addition to our novel solution approach for SSAP-RA, we also attempt to fill the gap in the literature mentioned by Bianchi-Aguiar et al. (2021) by considering the rectangular arrangement of products in our exact solution methods.

We present the details of SSAP-RA and our mixed-integer programming model (MIP) in the next section.

2. Shelf space allocation problem with rectangular arrangement (SSAP-RA) and mixed integer linear programming model (MIP)

The motivation for our study originates from a real-world application based on a local bookstore. The bookstore managers aim to find an arrangement of (available) books on shelves to increase their sales, hence improving their profits. Due to legal and trade agreements, the bookstore needs to obtain books from different publishers in various quantities, and hence, the bookstore cannot necessarily choose the exact number and title of books it receives, i.e., the bookstore does not do assortment decisions. To improve the accessibility and visibility of books, the bookstore managers arrange the same books together when there is more than one facing unit and all of this is achieved by rectangular arrangement. However, they do not have a systematic way to determine a good facing/space allocation strategy for their books and they make these decisions based on their past experiences and previous allocations. Therefore, we aim to solve the books' allocation problem in rectangular arrangement to maximize profit, defined as a SSAP-RA.

SSAP-RA is the allocation of products to shelves to maximize profit by placing the products' facing units in two dimensions (x-dimension/width and y-dimension/height) and providing a rectangular arrangement. The number of facing units in two dimensions (i.e., number of units of a product and their location on the shelves) are decisions. Indices, parameters, sets, and decision variables are given in Table 3.

The set of products is represented with $D=\{1,2,\dots,D_{\max}\}$ and the set of cabinets is given by $K=\{1,2,\dots,K_{\max}\}$. Product $d \in D$ is displayed in a cabinet $k \in K$. The total space/area that we can assign and allocate products is represented with R_k and C_k parameters. The cabinet k includes multiple shelves and has R_k number of rows and C_k columns; therefore, a cabinet has $R_k C_k$ slots capacity (from now on, we refer shelves as rows). A slot is the place that we assign for the thinnest product, and one product facing may occupy more than one consecutive slot. The total number of columns C_k is calculated by the total number of slots that depend on the products' thicknesses (see Eqs. (1) and (2)). All facings of a product must be assigned to the same cabinet providing a rectangular arrangement.

The decision variables that include the location of a product are x_{dk} , y_{dik} , z_{djk} , and t_{djk} . With these binary decision variables, we can ensure rectangular arrangement of products. The size of the rectangular arrangement of product d is determined by integer decision variables; w_d , h_d , m_{dk} , n_{dik} , and p_{djk} . The total number of facings of product d at cabinet k is represented by m_{dk} . This decision variable m_{dk} is the summation of n_{dik} over each row for each product

Table 3

Notation.

Sets:	
D	The set of products. $D=\{1,2,\dots,D_{\max}\}$.
K	The set of cabinets. $K=\{1,2,\dots,K_{\max}\}$.
I	The set of rows. $I=\{1,2,\dots,I_{\max}\}$.
J	The set of columns. $J=\{1,2,\dots,J_{\max}\}$.
Indices:	
d	Index of products to be allocated, $d \in D$.
k	Index of cabinet, $k \in K$.
i	Index of rows, $i \in I$.
j	Index of columns, $j \in J$.
Parameters:	
R_k	The number of rows in cabinet k .
C_k	The number of columns in cabinet k .
A_i	The effectiveness of row i .
V_d	The profit of product d .
L_d	The lower bound of product d .
U_d	The upper bound of product d .
T_d	The thickness of product d .
Decision Variables:	
x_{dk}	$\begin{cases} 1, \text{ if product } d \text{ is assigned to cabinet } k; \\ 0, \text{ otherwise.} \end{cases}$
y_{dik}	$\begin{cases} 1, \text{ if product } d \text{ is assigned to row } i \text{ at cabinet } k; \\ 0, \text{ otherwise.} \end{cases}$
$z_{dj k}$	$\begin{cases} 1, \text{ if product } d \text{ is assigned to column } j \text{ at cabinet } k; \\ 0, \text{ otherwise.} \end{cases}$
t_{dijk}	$\begin{cases} 1, \text{ if product } d \text{ is assigned to row } i \text{ and column } j \text{ at cabinet } k; \\ 0, \text{ otherwise.} \end{cases}$
m_{dk}	Total number of facings of product d at cabinet k .
n_{dik}	Number of facings of product d in row i at cabinet k .
$p_{dj k}$	Number of facings of product d in column j at cabinet k .
w_d	Width of product d .
h_d	Height of product d .

and cabinet where n_{dik} are the number of facings of product d in row i at cabinet k . Variables $p_{dj k}$ show the number of facings of product d in column j at cabinet k . The width and height of the rectangular shape of product d (also called the width and height of product d) are represented by decision variables w_d and h_d consecutively. If product d is assigned to more than one row, these rows must be consecutive, and the number of facings of product d at each consecutive row should be the same, which are represented by the *width* of product d , w_d . Furthermore, if product d is allocated to more than one slot/column in a row, these columns must be consecutive for each row, and the number of facings of product d at consecutive columns should be the same, which are represented by the *height* of product d , h_d . Thus, all facings of product d are placed in an adjacent $w_d h_d$ rectangular shape. U_d and L_d are the upper bound and the lower bound of facings of product d , and they are given problem parameters.

Fig. 1 shows an example of a cabinet in a bookstore with the rectangular display of 26 books with different number of facing units. Book 1 has three facings in a row (row 1), $w_1 = 3$ and $h_1 = 1$, which are displayed consecutively. Book 7 has six facings located in rectangular arrangement with $w_7 = 3$ and $h_7 = 2$. Book 23 has nine facings located as a rectangular arrangement with $w_{23} = 3$ and $h_{23} = 3$.

Based on our local bookstore interactions and requirements, each product has a different thickness represented by parameter T_d . To handle varying levels of thicknesses, we have to include T_d to our model. This modeling addition changes the problem significantly because we consider the location of products in a discrete manner, as our problem requires, and also done in some of the studies in the literature (Geismar et al., 2015). The thinnest product thickness is taken as 1, and other products' thickness values are determined according to the ratio between them. For instance, let s_d represent the space that one facing unit of product d occupies at a row, and the thinnest one at the same cabinet is product

e with space s_e for one facing unit. The thickness of product e is taken as 1, $T_e = 1$, and the thickness of product d is determined by the ratio, $T_d = \lceil (s_d/s_e) \rceil$, which is rounded up. Using this ratio we can find all products' thicknesses that lead us to calculate a lower bound for the corresponding problem's total number of columns.

The number of columns in a cabinet depends on the length of a row and the space of the thinnest product's one facing unit (s_e). To calculate the number of columns for any given problem instance, we develop an approach that considers the instance size. To find a lower bound for the columns in a problem instance, we calculate the c^* value that allocates at least f times of the lower bounds of facings of all products, as in Eq. (1). Recall that L_d is the lower bound of product d and T_d is the thickness of product d . The f value is estimated based on problem instances. Because we solve a SSAP-RA, we have to consider the c^* in two dimensions. Therefore, we find the lower bound of C_k (the number of columns in cabinet k) by rounding up the c^* to the nearest integer, which is a multiple of R_k , as in Eq. (2). We take that $R_k = 8$ for all instances, which is equal to the number of rows of cabinets commonly used in bookstores.

$$c^* = \lceil (\sum_{d \in D} L_d * T_d * f) \rceil \quad (1)$$

$$\sum_{k \in K} C_k = \frac{c^*}{R_k} + (1 - \frac{c^* \bmod R_k}{R_k}) \quad (2)$$

In infeasible instances, we increase the c^* parameter incrementally. Since we generate feasible size of cabinets considering two dimensions, the rectangular requirement does not result any prime number products, which may occur if one dimensional solutions are transferred to a two-dimensional setting (Hübner et al., 2021). Since the upper bounds of products, U_b , are smaller than the number of columns in a row, our models can assign any product with prime number facings in one row sequentially. Although products'

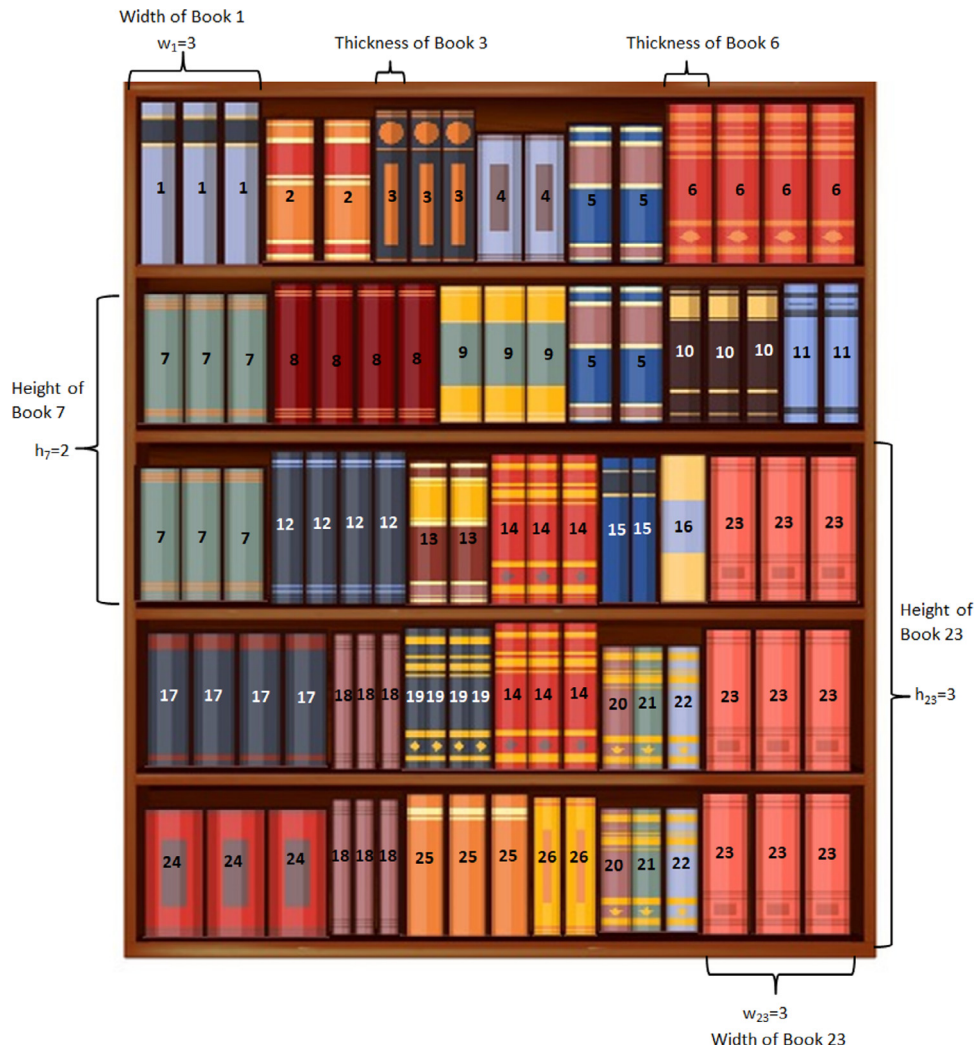


Fig. 1. A sample allocation of books in a cabinet.

heights may also differ, it does not pose a challenge since the distance between two rows/shelves is higher than the height of a product.

As discussed before, we assume that demand is only affected by space-elasticity and location-elasticity, and we omit horizontal effects and cross-elasticities since the literature gives more importance to space and location elasticities, and horizontal effects and cross-elasticities are harder to estimate or they are not available most of the time (Brown & Lee, 1996; Chandon et al., 2009; Dreze, Hoch, & Purk, 1994; Kök et al., 2015). We simplify our demand function using the ideas in Corstjens & Doyle (1981) and ignore cross-elasticities motivated by the literature (Bianchi-Aguilar et al., 2018; Yang & Chen, 1999). We define our demand function $Q_{dik}(n_{dik})$ as in Eq. (3) where n_{dik} is the number of facings of product d displayed on row (shelf) i at cabinet k , α_d is the unmodified base demand, which is equal to the minimum demand if product d had one facing unit (which is 1 in our case), and β_{dik} is the space-elasticity (Borin, Farris, & Freeland, 1994; Corstjens & Doyle, 1981). To be consistent with our terminology, we also included the cabinet index k to our demand function.

$$Q_{dik}(n_{dik}) = \alpha_d * n_{dik}^{\beta_{dik}} \quad (3)$$

The objective function depends on the location of products and aims to maximize profit by changing the vertical position of products and their number of facings. Given that the profit obtained

from product d is V_d , the objective function can be modeled using the S-shaped Gompertz growth model, which has a non-linear characterization (Dreze et al., 1994), as presented in Eq. (4).

$$z = V_d * \alpha_d * n_{dik}^{\beta_{dik}} \quad (4)$$

To simplify Corstjens & Doyle (1981) study, Yang & Chen (1999) assumed that product profit was linear within a small number of facings. They proposed to use an upper bound and a lower bound of facings for product d (U_d and L_d) to ensure the approximation is valid when it is not possible to estimate β_{dik} . Therefore, the objective function becomes linear and the vertical location impact is captured by the effectiveness of each row i , A_i , as in Eq. (5).

$$z = V_d * A_i * \alpha_d * n_{dik} \quad (5)$$

The potential profit z (in row i for product d in cabinet k) is calculated by multiplying V_d (profit for product d), A_i (effectiveness of row i), α_d (minimum demand if product d had one facing unit, which is 1 in our case) and n_{dik} (the number of facings of product d displayed on row i at cabinet k). A_i is the highest in the middle of a cabinet, and it decreases towards lower and upper rows. As mentioned above, the rows' height is higher than the tallest product's height, and empty slots are not allowed.

To solve our problem we develop a mixed-integer linear programming model (MIP), a logic-based Benders decomposition model (LBBDD1) (Section 4.1), and an exact 2-stage iterative solution

approach (IP1/IP2) (Section 4.2). The notation used in the models is often the same. The model-specific variables are explained in the relevant sections. We next present the MIP model.

$$\max \sum_{d \in D} \sum_{i \in I} \sum_{k \in K} V_d A_i n_{dik} \quad (6)$$

$$\sum_{k \in K} x_{dk} = 1 \quad \forall d \in D \quad (7)$$

$$\sum_{d \in D} t_{dijk} = 1 \quad \forall i \in I, \forall j \in J, \forall k \in K \quad (8)$$

$$\sum_{j \in J} t_{dijk} = n_{dik} T_d \quad \forall d \in D, \forall i \in I, \forall k \in K \quad (9)$$

$$\sum_{i \in I} n_{dik} = m_{dk} \quad \forall d \in D, \forall k \in K \quad (10)$$

$$\sum_{i \in I} t_{dijk} = p_{dj k} \quad \forall d \in D, \forall j \in J, \forall k \in K \quad (11)$$

$$x_{dk} L_d \leq m_{dk} \quad \forall d \in D, \forall k \in K \quad (12)$$

$$x_{dk} U_d \geq m_{dk} \quad \forall d \in D, \forall k \in K \quad (13)$$

$$\sum_{d \in D} n_{dik} T_d = C_k \quad \forall i \in I, \forall k \in K \quad (14)$$

$$\sum_{d \in D} p_{dj k} = R_k \quad \forall j \in J, \forall k \in K \quad (15)$$

$$w_d - n_{dik} T_d \leq C_k (1 - y_{dik}) \quad \forall d \in D, \forall i \in I, \forall k \in K \quad (16)$$

$$n_{dik} T_d \leq C_k y_{dik} \quad \forall d \in D, \forall i \in I, \forall k \in K \quad (17)$$

$$h_d - p_{dj k} \leq R_k (1 - z_{dj k}) \quad \forall d \in D, \forall j \in J, \forall k \in K \quad (18)$$

$$p_{dj k} \leq R_k z_{dj k} \quad \forall d \in D, \forall j \in J, \forall k \in K \quad (19)$$

$$n_{dik} T_d \leq w_d \quad \forall d \in D, \forall i \in I, \forall k \in K \quad (20)$$

$$\sum_{i \in I} y_{dik} \leq h_d \quad \forall d \in D, \forall k \in K \quad (21)$$

$$y_{d,i+1,k} + w_d \geq y_{dik} + n_{d,i+u,k} T_d \quad \forall d \in D, \forall i \in I, \forall k \in K, \quad (22)$$

$$\forall u \in I \mid 1 \leq u; \text{ \& } u \leq I_{\max} - 1; \text{ \& } i + u \leq I_{\max}$$

$$z_{d,j+1,k} + h_d \geq z_{dj k} + p_{d,j+u,k} \quad \forall d \in D, \forall j \in J, \forall k \in K, \quad (23)$$

$$\forall u \in J \mid 1 \leq u; \text{ \& } u \leq J_{\max} - 1; \text{ \& } j + u \leq J_{\max}$$

$$x_{dk}, y_{dik}, z_{dj k}, t_{dijk} \in \{0, 1\}, \quad \forall d \in D, \forall i \in I, \forall j \in J, \forall k \in K \quad (24)$$

$$m_{dk}, n_{dik}, p_{dj k}, w_d, h_d \in \{0, \mathbb{Z}^+\} \quad \forall d \in D, \forall i \in I, \forall j \in J, \forall k \in K \quad (25)$$

The objective function of MIP (6) maximizes the profit of displayed products. Constraint (7) ensures that each product is assigned to only one cabinet. Constraint (8) assigns exactly one facing unit to each slot. Constraint (9) computes the number of facings of product d in row i , n_{dik} . Constraint (10) calculates the total number of

facings in two dimensions of product d in Cabinet k , m_{dk} . Constraint (11) determines the number of facings of product d in column j , $p_{dj k}$. (These decision variables are auxiliary variables that provide communication between models detailed in the next sections.) Constraints (12) and (13) restrict the number of assigned facings of all products with respect to their lower bounds (L_d) and upper bounds (U_d). Constraints (14) and (15) limit the display quantities of products with the dimensions of the cabinet that contains C_k columns and R_k rows. Constraints (16) and (17) ensure that if product d exists in a row, the number of facings of product d should be equal to the width of product d , w_d . Constraints (18) and (19) make sure that the number of facings of a product in different columns is equal to each other, h_d . Constraints (20) and (21) determine the width w_d and height h_d variables of product d . Constraint (22) guarantees that the facings of a product in a row are allocated consecutively. Similarly, constraint (23) provides that the facings of a product in a column are allocated consecutively. Therefore, within these constraints, the model can ensure rectangular arrangement of products. Lastly, the expressions (24) - (25) determine the solution space of the decision variables.

Since the consecutive allocation of facings in columns has no contribution to the objective function of SSAP-RA, most of the studies in the literature ignore the column sequencing in their solution approach and apply a column sequencing approach separately when needed by the application of SSAP-RA. In our MIP model, we achieve rectangular arrangement of facings for all products by providing the consecutive allocation of facings of the product in rows and columns jointly and simultaneously.

SSAP-RA is an NP-Hard problem since it belongs to the class of two-dimensional knapsack problems. The two-dimensional knapsack problems guarantee the arrangement of a set of rectangles $r \in R$ to a capacitated two-dimensional rectangular container S with a specific width (S^{width}) and depth (S^{depth}) (Hübner et al., 2021). In our problem, the cabinet at the bookstore is a capacitated two-dimensional rectangular container K with a certain width (C_k) and height (R_k). The rectangle r represents not the item itself but its facings which have two dimensions, width and height. We can omit the boundaries of inner shelves because these boundaries do not restrict the allocation of the rectangles and their dimensions. Also, our problem does not include any constraint about inner shelf height. Therefore, regardless of the boundaries, the cabinet becomes a capacitated two-dimensional rectangular container. Furthermore, another challenging part of our problem compared to two-dimensional knapsack is that the dimensions of rectangles are unknown initially and our models need to determine the width and height of the items.

To solve our MIP efficiently, we develop two solution approaches and present them in the next section. After we discuss our solution methods, we compare the performances of MIP and two solution approaches in Section 5 using two different datasets. The first dataset (*RealisticData*) is based on a real-world dataset (*CaseData*) provided by our local bookstore, and the second one (*SyntheticData*) is generated by the specific distributions proposed by Geismar et al. (2015). To mimic the real-world allocations at the bookstore we develop three different scenarios based on *CaseData* to provide a profit improvement estimate using our approach for the bookstore.

3. Solution approaches

The experiments with different datasets showed that our MIP model could not solve realistic instances in acceptable times. To solve our MIP model efficiently, we develop two solution approaches. Our first method is based on logic-based Benders decomposition (LBBD), and the second one is a novel iterative method inspired by our LBBD method and it includes two simpler mathematical programs.

3.1. The LBB based solution method

LBB is a generalization of classical Benders decomposition for large-scale optimization problems (Hooker & Ottosson, 2003). A problem is partitioned into two parts; *master problem* (contains only hard variables), and *sub-problem*. The master problem finds tentative values of the hard variables and sends this information to sub-problem to find a feasible solution. If the sub-problem solution is infeasible, a *Benders feasibility cut* is added to the master problem to eliminate the current solution. If the sub-problem solution is feasible but not optimal, a *Benders optimality cut* is added to the master problem to guarantee convergence to an optimal solution. Therefore in both cases, better tentative values for the hard variables can be found, and the algorithm continues until it reaches an optimal solution (Hooker, 2019).

Benders cuts are the fundamental tools for Benders decomposition (Chu & Xia, 2004). In classical Benders decomposition, a cut is generated by the weak duality property of linear programming. However, this property cannot be used in integer programming, and it becomes harder to generate a *valid cut*.

To solve SSAP-RA, we first have to determine optimal number of facings of all products in two dimensions, and then ensure rectangular arrangement of them. Since number of facings of product d (decision variables w_d and h_d) are unknown at the beginning, the shelf space allocation problem is challenging to solve. If the number of facings of products (w_d and h_d ; $\forall d \in D$) were known, the problem becomes allocation and sequencing problem of products with different rectangular sizes on a finite space.

Therefore, there are two main actions to be performed to solve the problem: 1) Finding optimal number of facings of products, and 2) Allocating facings of products in rectangular arrangement. The master problem ignores rectangular arrangement of facings and considers only finding optimal number of facings of products. Then, it sends this information to the sub-problem. The sub-problem allocates and sequences the facings of products to maximize the profit without changing their w_d and h_d values. The algorithm continues until it reaches an optimal solution.

Although we separate our problem into two simpler problems, the master problem and the sub-problem both solve a two-dimensional shelf space allocation problem because both of the models consider the width (w_d) and height (h_d) of the products. However, the master problem ignores rectangular arrangement while the sub-problem ensures it.

We denote our LBB based solution method as LBB1. In LBB1, the master problem is modeled by MIP1 which is a simplified version of MIP and the sub-problem is modeled by SMIP1 with tentative assignments (MIP1-SMIP1). We need to define additional notation for LBB1. Assume that $S_A = \{< d, i, j, k > | d \in D, i \in I, j \in J, k \in K\}$, $S_{dijk}^h = \{S_A | t_{dijk} = 1 \text{ at iteration } h\}$, and M is a large number. We also define the fixed values of decision variables of MIP1 as parameters for SMIP1, i.e., w_d^{MIP1} , h_d^{MIP1} , and m_{dk}^{MIP1} . MIP1 includes constraints (6)–(21), and the definition space constraints (24) and (25). Also, we add constraints (26) and (27) which represent the Benders feasibility cuts and Benders optimality cuts, respectively. These cuts in constraints (26) and (27) are called *weak cuts* (Hooker, 2007), and we prove their optimality in Appendix A.

$$\sum_{S_{dijk}^h} (1 - t_{dijk}) \geq 1 \quad (26)$$

$$Z_{MIP1} \leq Z_{SMIP1} * [1 - \sum_{S_{dijk}^h} (1 - t_{dijk}) + M * \sum_{S_{dijk}^h} (1 - t_{dijk})] \quad (27)$$

The sub-problem SMIP1 is the same with the original model MIP that contains all constraints (6)–(25) and (28)–(30). The

fixed/tentative w_d^{MIP1} , h_d^{MIP1} , and m_{dk}^{MIP1} variables, sent by MIP1, are added to SMIP1 with constraints (28)–(30). The objective functions of MIP1 and SMIP1 are the same with the MIP model.

$$w_d^{SMIP1} = w_d^{MIP1}, \quad \forall d \in D \quad (28)$$

$$h_d^{SMIP1} = h_d^{MIP1}, \quad \forall d \in D \quad (29)$$

$$m_{dk}^{SMIP1} = m_{dk}^{MIP1}, \quad \forall d \in D, \forall k \in K \quad (30)$$

The pseudo-code of LBB1 is presented in Algorithm 1. In the

Algorithm 1: LBB1 pseudo-code.

```

Input :  $A_i, V_d, U_d, L_d, T_d, R_k, C_k$ 
Output: CPUtime,  $Z_{MIP1}$ ,  $Z_{SMIP1}$ , LB,  $t_{dijk}^{SMIP1}$ 
1 Initialization:  $LB = -\infty$ ;
2 Incumbent  $\leftarrow \{LB, -\}$ ;
3 While Status = 0 do
4   Solve MIP1, compute variables
    $w_d^{MIP1}, h_d^{MIP1}, m_{dk}^{MIP1}, x_{dk}^{MIP1}, n_{dik}^{MIP1}, y_{dik}^{MIP1}, t_{dijk}^{MIP1}$ ;
5   if MIP1 is infeasible then
6     Status  $\leftarrow 1$ ;
7   else
8      $Z_{MIP1} \leftarrow$  Objective value of MIP1;
9      $w_d^{SMIP1} = w_d^{MIP1}$ ;
10     $h_d^{SMIP1} = h_d^{MIP1}$ ; // Set w, h, m values in sub-problem SMIP1
11     $m_{dk}^{SMIP1} = m_{dk}^{MIP1}$ ;
12    Solve SMIP1;
13    if SMIP1 is feasible then
14       $Z_{SMIP1} \leftarrow$  Objective value of SMIP1;
15      Update LB,  $LB = \max(Z_{SMIP1}, LB)$ ;
16      if  $Z_{MIP1} = Z_{SMIP1}$  then
17        Incumbent  $\leftarrow \{LB, t_{dijk}^{SMIP1}\}$ ;
18        Status  $\leftarrow 1$ ;
19      else if CPUtime  $\geq$  Time.limit then
20        Incumbent  $\leftarrow \{LB, t_{dijk}^{SMIP1}\}$ ;
21        Status  $\leftarrow 1$ ;
22      else if  $|Z_{MIP1} - LB| / Z_{MIP1} \leq \epsilon$  then
23        Incumbent  $\leftarrow \{LB, t_{dijk}^{SMIP1}\}$ ;
24        Status  $\leftarrow 1$ ;
25      else if  $|Z_{MIP1} - LB| / Z_{MIP1} > \epsilon$  then
26        Status  $\leftarrow 1$ ;
27        Generate optimality cuts (27);
28    else
29      Generate feasibility cuts (26);
30    end
31    Add cuts to master problem;
32  end
33 exit

```

first LBB1 iteration, we run MIP1 without Benders cuts and our first solution Z_{MIP1} will be an *upper bound* of the objective function. Since the objective function only depends on the assigned row of products, it is not affected by the assigned column while the row stays the same. Therefore, we are able to use the same objective function for the master problem and the sub-problem. The LBB1 algorithm solves MIP1 to determine the w_d , h_d , and m_{dk} variables, and then, sends these tentative assignments to SMIP1. SMIP1 starts with fixed/tentative values of w_d , h_d , and m_{dk} variables and tries to place the facings of a product in rectangular arrangement, namely sequencing facings in rows and columns.

Three different outcomes can happen when the LBB1 continues after a feasible MIP1 solution:

- The objective function values of two models are equal; $Z_{MIP1} = Z_{SMIP1}$. Then, the current solution is a global optimum and we terminate the LBB1 algorithm.
- The models reach their optimal solutions, but they are not equal to each other; $Z_{MIP1} > Z_{SMIP1}$. If the termination condition determined by the user is satisfied ($|Z_{MIP1} - Z_{SMIP1}| / Z_{SMIP1} \leq \epsilon$),

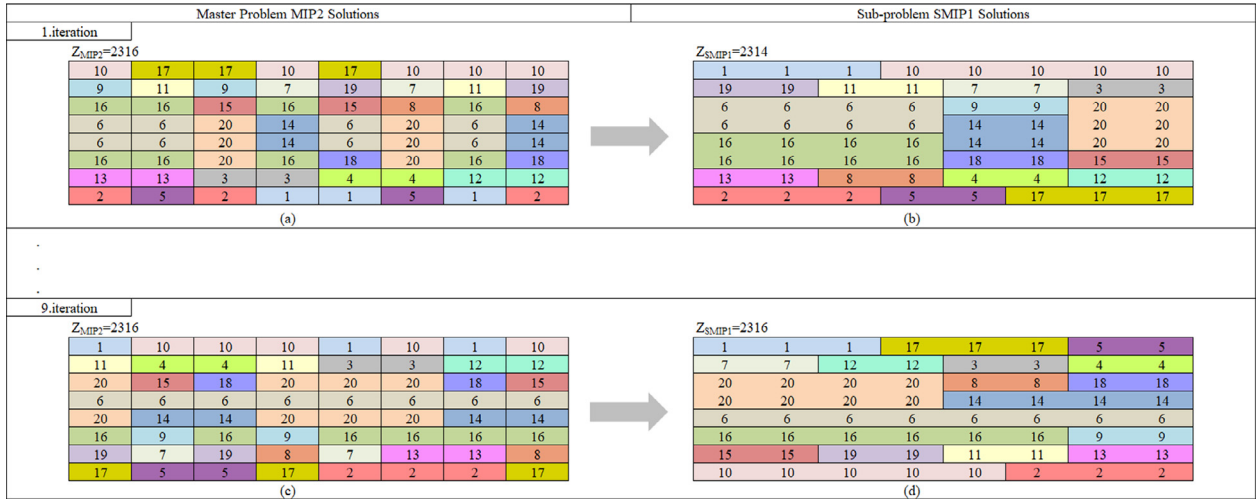


Fig. 2. The solution of an instance with the LBBD1 algorithm.

we terminate the LBBD1 algorithm. Otherwise, we add Benders optimality cuts to the master problem MIP1. We take $\epsilon = 0.5\%$.

- The sub-problem SMIP1 solution is infeasible. In this case, we add Benders feasibility cuts to the master problem MIP1.

If the current assignments are infeasible, then we utilize the feasibility cut (26) to switch at least one t_{dijk} to zero. Therefore, MIP1 changes the tentative assignments and the current solution is eliminated from the solution space. The Benders optimality cuts consider this set and construct the constraint (27). If the tentative assignments between iterations do not change (i.e., the current assignment is feasible), the objective function of the master problem Z_{MIP1} is restricted by the objective function of the sub-problem Z_{SMIP1} ; otherwise (i.e., if current assignments are infeasible, then they do change) this constraint (27) will be redundant.

An example solution of LBBD1 is shown in Fig. 2 that contains the arrangement of 20 products with different facings optimally to 8x8 cabinet. At the first iteration, MIP1 finds the allocation of products at Fig. 2(a), and then after the sub-problem SMIP1 runs with tentative assignments, the assignment in Fig. 2(b) is found. Since $Z_{MIP1} > Z_{SMIP1}$, iterations are continued with Benders optimality cuts. At iteration 9, the assignment in Fig. 2(d) provides $Z_{MIP1} = Z_{SMIP1}$ and LBBD1 terminates.

LBBD1 can handle larger instances compared to MIP, however, it also falls short to solve some of the realistic instances of the problem (see the next section). Inspired by our LBBD1 model, we develop an exact and efficient 2-stage iterative solution approach, IP1/IP2, which separates row and column consecutiveness conditions, detailed in the next section.

3.2. Exact 2-stage iterative solution approach: IP1/IP2 method

To fasten the LBBD1 algorithm, we separate the MIP model into two simple models, IP1 and IP2. IP1 includes all the constraints in MIP (6)–(25) except (23), which ensures the consecutive allocation of facings of a product in a column. First IP1 runs and it sends tentative assignments of decision variables w_d , h_d , x_{dk} , y_{dik} , m_{dk} , n_{dik} to IP2. IP2 contains all constraints of MIP, including the constraint (23), and the fixed values of decision variables sent by IP1. Then IP2 determines the z_{djk} and p_{djk} decision variables and finds optimal values of t_{dijk} variables, particularly the optimal locations of facings of all products. The pseudo-code of the IP1/IP2 is given in Algorithm 2. Unlike the LBBD1 model, IP1/IP2 method does not use Benders cuts and does not compare the objective functions of two models, and runs only one iteration. The objective func-

Algorithm 2: IP1/IP2 method pseudo-code.

Input : $A_i, V_d, U_d, L_d, T_d, R_k, C_k$
Output: CPUtime, Z_{IP2} , t_{dijk}^{IP2}

- 1 Solve IP1, compute variables $w_d^{IP1}, h_d^{IP1}, m_{dk}^{IP1}, x_{dk}^{IP1}, n_{dik}^{IP1}, y_{dik}^{IP1}, t_{dijk}^{IP1}$.
- 2 **if** IP1 is feasible & CPUtime $\leq$ Time.limit **then**
- 3 $Z_{IP1} \leftarrow$ Objective value of IP1;
- 4 $w_d^{IP2} = w_d^{IP1}$;
- 5 $h_d^{IP2} = h_d^{IP1}$;
- 6 $m_{dk}^{IP2} = m_{dk}^{IP1}$; // Set w, h, m, n, x, y values in IP2
- 7 $n_{dik}^{IP2} = n_{dik}^{IP1}$;
- 8 $y_{dik}^{IP2} = y_{dik}^{IP1}$;
- 9 $x_{dk}^{IP2} = x_{dk}^{IP1}$;
- 10 Solve IP2;
- 11 **if** CPUtime $\leq$ Time.limit **then**
- 12 $Z_{IP2} \leftarrow$ Objective value of IP2;
- 13 Incumbent $\leftarrow (Z_{IP2}, t_{dijk}^{IP2})$;
- 14 **end**
- 15 **end**

tion and optimal t_{dijk} values are determined by the IP2 model. We prove the optimality of IP1/IP2 model in Appendix B.

Fig. 3 shows the IP1/IP2 solution of the same instance in Fig. 2. Different from LBBD1, IP1/IP2 runs only one iteration. At first, we solve the IP1 problem and ensure the consecutive allocation of facings of a product in a row. Then, we solve IP2 problem and ensure the rectangular arrangement of facings of all products. For instance, in Fig. 3(a), the facings of product 20 are allocated at consecutive rows. After performing IP2, as in Fig. 3(b), the facings of product 20 are located at consecutive rows and columns to ensure the rectangular arrangement. Since the objective function in constraint (6) is not affected by the column location of the facing units, the objective functions of IP1 and IP2 models are the same, $Z_{IP1} = Z_{IP2} = 2316$, which is also the optimal solution of IP1/IP2.

We next present and compare the performance of MIP, LBBD1 and IP1/IP2. We also explain datasets used in these comparisons and provide a discussion on our results.

4. Datasets

This section presents three datasets used for the performance evaluation of solution methods and the real-world application comparison. The first dataset *CaseData* is the actual data gathered from the local bookstore. The two other datasets were developed from different sources: the *RealisticData* is generated from

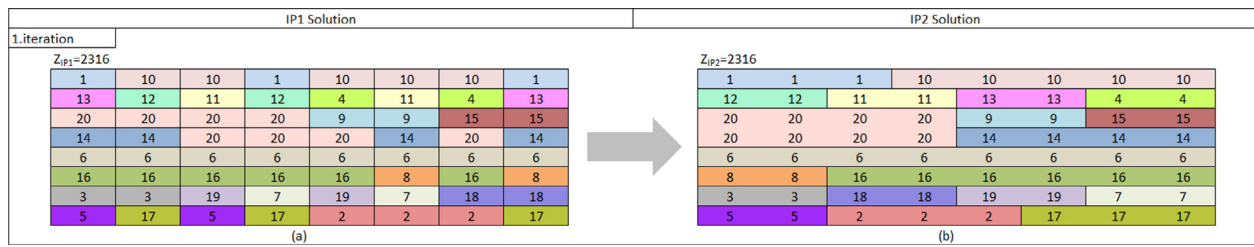


Fig. 3. The solution of an instance with the IP1/IP2 algorithm.

the *CaseData*, and the *SyntheticData* is generated from the literature benchmarks designed by Geismar et al. (2015). Since the *CaseData* is used for the generation of the *RealisticData*, we first explain the *CaseData* in the next section.

4.1. Real-world dataset from the bookstore: *CaseData*

We first describe the dataset obtained from the local bookstore, *CaseData*. To examine the effects of our proposed solution methods more clearly, we consider the *bestseller* section. The *bestsellers'* location contains four cabinets and includes 557 books in total; cabinet 1 has 134 books, cabinet 2 has 133 books, cabinet 3 has 136 books, and cabinet 4 has 154 books. We gather the names, the number of pages, the profits, the publisher of books, the number of facings, and the books' current placement (we compare the bookstore's current configuration exactly with our optimized plan and quantify savings in 5.3). The minimum and the maximum number of facings of books are 1 and 4, respectively. The number of pages of a book is used to determine its thickness. We find the thinnest and the thickest books and construct the thickness parameter, which is varied between 1 and 7. The cabinets have $R_k = 8$ ($k = 1, 2, 3, 4$) rows (shelves). We calculate the number of columns regarding the total area used for books at the current arrangement. Although the bookstore does not prefer empty shelf space in general, we observed that they cannot utilize the cabinet capacity at all times. *CaseData* was obtained as a snapshot of the real system, and it includes empty spaces. Since some of the rows have empty spaces, the number of used columns are different for cabinets, such as $C_1 = 47$, $C_2 = 52$, $C_3 = 51$, and $C_4 = 49$. The row effectiveness values A_i are inspired by literature. Dreze et al. (1994) showed that the eye-level rows are more important than the others and affect profit maximization. Therefore, we choose $A_i = (a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8) = (5, 6, 7, 8, 8, 7, 6, 5)$, whilst the eye-level rows have the highest impact value (row 4 and row 5), the extreme rows have the lowest (row 1 and row 8). We also use the same A_i parameter for the *RealisticData* and *SyntheticData*. With the help of this dataset, we determine the frequencies of parameters and generate a new dataset *RealisticData* that includes larger instances, detailed in the next section.

4.2. Generated dataset from the *CaseData*: *RealisticData*

We first analyze the data from the bookstore (*CaseData*) statistically and determine the frequencies of parameters (on the thickness of books and profitability of the books), which are listed in Table 4. Then, we use these empirical probabilities of the parameters to generate instances randomly and we call these instances *RealisticData*.

We construct 48 test instances with different numbers of books, between 10 and up to 250 books. The number of rows is 8 for all instances, and the number of columns varies between 8 - 160 columns, calculated by Eqs. (1) and (2). The *RealisticData* instances are available at Mendeley data (Caglar Gencosman & Begen, 2021). Some of the instances are run twice with different cabinets; thus,

Table 4

Empirical distribution of parameters.

Thickness of books	Probability	Profit of Books	Probability
1	0.072	3–5	0.074
2	0.530	5–7	0.214
3	0.264	7–9	0.206
4	0.106	9–11	0.199
5	0.020	11–13	0.235
6	0.004	13–15	0.057
7	0.005	15–17	0.007
		17–19	0.002
		19–21	0.002
		21–24	0.004

we run 55 different experiments. Next, we describe the *SyntheticData* dataset.

4.3. Generated dataset from the literature: *SyntheticData*

To determine the performance of our methods in a more generic way, we construct *SyntheticData* by using the benchmarks from the literature. We use the distribution functions of parameters proposed by Geismar et al. (2015). The profit parameter is normally distributed $N(12, 8)$, the lower bounds of the facings are assumed to be 1, and the upper bounds are determined by $U_d = \lceil r_d / (\sum_{h \in D} r_h) * (R_k * C_k * K) \rceil$, where r_d is the profit of book d , R_k and C_k values represent the capacity of cabinet K . (Note that, in Geismar et al. (2015), r_d represents the revenue of book d). We use the empirical distribution obtained from our local bookstore for the thicknesses of books. To ensure the feasibility of solutions, we determine the number of columns in the instances appropriately with Eqs. (1) and (2).

We construct 48 test instances with different numbers of books, between 10 and up to 225 books. The number of rows is 8 for all instances, and the number of columns varies between 8 - 142 columns, calculated by Eqs. (1) and (2). The *SyntheticData* instances are available at Mendeley data (Caglar Gencosman & Begen, 2021).

We run our models on 100 instances from *RealisticData* (48 instances) and *SyntheticData* (48 instances) to examine proposed methods' performances. Also, we use *CaseData* (4 instances) to provide managerial insights with three different scenarios for the real-world application in the next section.

5. Results

We present and compare the performance of our MIP and two solution approaches for SSAP-RA. The models are implemented in IBM ILOG CPLEX Optimization Studio 12.8.0, and all computations are performed on a machine with Windows 10 (64-Bit) operating system, 8 GB RAM and an Intel Core i7-9750H CPU with 2.59 GHz. We use the clock time as a termination condition and set it 1800 seconds in all experiments with 100 test instances.

Table 5Comparison results of developed solution methods within *RealisticData*.

Prob.	# of Books	# of Cols.	# of Cabs.	MIP1			LBBD1				IP1/IP2	
				Obj.	Time	%GAP	Obj.	Time	%GAP	# of Cuts	Obj.	Time
R1	10	8	1	2260	2.0	0	2260	5.5	0	4	2260	1.9
R2		8	1	1996	3.2	0	1996	2.2	0	0	1996	1.9
R3		8	1	1652	41.1	0	1638	1800	0.009	508	1652	4.2
R4	20	8	1	2241	155.8	0	2241	7.2	0	0	2241	4.2
R5		8	1	2316	24.2	0	2316	3.1	0	0	2316	2.1
R6		8	1	2432	300.3	0	2432	7.1	0	3	2432	2.1
R7	30	12	1	3646	314.9	0	3646	4.5	0	0	3646	2.3
R8		12	1	2965	338.1	0	2965	8.5	0	0	2965	2.7
R9		12	1	2596	1800	7.5	2791	4.3	0	0	2791	2.5
R10	40	24	1	–	1800	–	–	1800	–	1	6534	6.5
R10		12	2	–	1800	–	6521	1800	0.002	1	6534	315.5
R11		24	1	–	1800	–	7171	1800	0.015	2	7278	3.9
R11		12	2	–	1800	–	7278	1263.8	0	5	7278	34.4
R12		24	1	–	1800	–	7444	1800	0.008	1	7504	5.3
R13	50	32	1	–	1800	–	–	1800	–	1	9823	8.8
R13		16	2	–	1800	–	–	1800	–	1	9823	139.8
R14		32	1	–	1800	–	–	1800	–	1	8584	6.1
R14		16	2	–	1800	–	–	1800	–	–	8584	44.4
R15		32	1	–	1800	–	–	1800	–	1	9258	5.6
R15		16	2	–	1800	–	–	1800	–	–	9258	173.9

Table 6IP1/IP2 performance for larger problems in *RealisticData*.

Prob.	# of Books	# of Cols.	# of Cabs.	Obj.	Time	Prob.	# of Books	# of Cols.	# of Cabs.	Obj.	Time
R16	60	40	1	10,884	9.0	R31	125	80	1	20,184	102.8
R16	60	20	2	10,884	382.7	R32	125	76	1	21,645	283.8
R17	60	40	1	10,473	23.9	R33	125	76	1	21,954	94.5
R18	60	40	1	11,539	9.69	R34	150	92	1	25,159	186.8
R19	70	40	1	10,540	10.6	R35	150	92	1	26,684	81.6
R20	70	40	1	10,259	43.1	R36	150	92	1	25,467	91.8
R20	70	20	2	10,259	223.8	R37	175	100	1	29,642	112.7
R21	70	40	1	10,687	11.2	R38	175	100	1	27,090	424.3
R22	80	50	1	12,406	71.5	R39	175	90	1	25,524	316.4
R23	80	50	1	13,099	16.9	R40	200	120	1	33,536	651.8
R24	80	50	1	13,902	51.6	R41	200	120	1	31,955	679.1
R25	90	56	1	15,116	24.1	R42	200	120	1	31,928	557.6
R26	90	60	1	15,879	71.7	R43	225	140	1	37,576	995.4
R27	90	56	1	15,185	52.4	R44	225	140	1	37,119	607.9
R28	100	60	1	16,891	69.3	R45	225	136	1	37,006	520.6
R29	100	64	1	17,266	40.3	R46	250	148	1	41,280	1228.4
R30	100	64	1	16,227	78.8	R47	250	160	1	42,461	1215.2
						R48	250	160	1	41,867	1121.3

5.1. Experiments with *RealisticData*

The first four columns at the [Tables \(5\)–\(8\)](#) give information about instances such as the number of books (*Books*), number of columns (*Cols.*), and number of cabinets (*Cabs.*). Recall that the number of rows parameter is set to 8 for all instances for both of the datasets. We increase the problem size (by number of books and columns) gradually and generate different instances (e.g., there are five different instances that include 40 books). In order to see the effect of the number of cabinets on the models' performances, some computations include the same instance with different numbers of cabinets. We detail each model's result by presenting the objective function value (*Obj.*), elapsed time in seconds (*Time*), and the GAP% provided by CPLEX as in [Eq. \(31\)](#). The gap value is calculated by the branch & cut algorithm in CPLEX 12.8, using the best integer solution z^* , and the best solution z' . We use this value as an indicator of how the current solution is far from the optimal value.

$$gap = \frac{|z^* - z'|}{e^{-10} + |z^*|} \quad (31)$$

[Tables 5](#) and [6](#) present the performance comparison of MIP, LBBD1 and IP1/IP2 with 48 test instances in the *RealisticData* dataset. We see that MIP is the slowest method. While it can solve instances up to 30 books optimally, it could not find any solution for instances with 40 books or more in the given time limit. LBBD1 is slightly better than MIP and can find optimal solutions of instances with 40 books. Nevertheless, neither MIP nor LBBD1 are helpful for the real-world instances of the bookstore SSAP-RA which have at least 133 books in each cabinet.

On the other hand, IP1/IP2 outperforms MIP and LBBD1 and reaches optimal solutions of small-size instances fast, as in [Table 5](#). IP1/IP2 finds the optimal solutions of problems (with one cabinet) up to 100 books less than 2 minutes. Moreover, the method provides optimal solutions of instances up to 250 books within time limit, as in [Table 6](#).

Although the test instances contain specific number of books, allocated facings are different. For example, the problem R47 includes 250 books, and IP1/IP2 allocates 571 facings in rectangular arrangement optimally; similarly, the problem R48 also contains 250 books and the IP1/IP2 method allocates 564 facings optimally

Table 7Comparison results of developed solution methods within *SyntheticData*.

Prob.	# of Books	# of Cols.	# of Cabs.	MIP1			LBBD1				IP1/IP2	
				Obj.	Time	%GAP	Obj.	Time	%GAP	# of Cuts	Obj.	Time
S1	10	8	1	4798	15.9	–	4798	5.9	0	–	4798	2.0
S2		8	1	4086	7.0	–	4086	2.7	0	–	4086	3.1
S3		8	1	2639	39.3	–	2639	20.1	0	–	2639	4.8
S4	20	8	1	3320	88.2	–	3320	54.5	0	6	3320	3.7
S5		8	1	3372	6.8	–	3372	3.0	0	–	3372	1.9
S6		8	1	3612	17.8	–	3612	3.4	0	–	3612	2.1
S7	30	20	1	–	1800	–	9613	1151.8	0	5	9613	10.7
S8		20	1	–	1800	–	11,118	18.84	0	–	11,118	21.7
S9		20	1	–	1800	–	7549	591.7	0	7	7549	10.7
S10	40	24	1	–	1800	–	11,721	1800	0.003	7	11,752	25.7
S11		24	1	–	1800	–	13,875	1800	0.002	1	13,908	31.4
S12		24	1	–	1800	–	10,545	1800	0.004	1	10,592	12.9
S13	50	32	1	–	1800	–	15,172	1800	0.039	1	15,763	39.8
S14		32	1	–	1800	–	16,479	1800	0.012	1	16,684	295.9
S15		28	1	–	1800	–	–	1800	–	–	14,450	32.3

Table 8IP1/IP2 performance for larger problems in *SyntheticData*.

Prob.	# of Books	# of Cols.	# of Cabs.	Obj.	Time	Prob.	# of Books	# of Cols.	# of Cabs.	Obj.	Time
S16	60	34	1	16,402	35.9	S32	100	60	1	27,661	30.7
S17	60	36	1	17,705	101.4	S33	100	60	1	29,928	150.6
S18	60	36	1	17,036	20.9	S34	125	78	1	28,993	186.6
S19	60	34	1	16,883	98.3	S35	125	72	1	29,192	903.7
S20	70	42	1	20,042	214.4	S36	125	72	1	32,214	629.4
S21	70	44	1	19,503	37.8	S37	150	80	1	31151	326.5
S22	70	42	1	18,379	35.6	S38	150	80	1	38675	361.9
S23	70	42	1	19,286	344.9	S39	150	80	1	31627	1657.1
S24	80	48	1	23,262	454.3	S40	175	90	1	41414	237.1
S25	80	48	1	23,586	45.1	S41	175	90	1	38542	374.1
S26	80	52	1	24,364	234.1	S42	175	90	1	40810	454.3
S27	80	48	1	27,308	306.6	S43	200	120	1	60509	1116.8
S28	90	52	1	21,178	148.7	S44	200	120	1	52951	1109.6
S29	90	52	1	27,269	526.7	S45	200	120	1	60780	1354.3
S30	90	52	1	26,938	747.7	S46	225	136	1	59390	1488.2
S31	100	60	1	30,774	55.6	S47	225	140	1	57,630	1348.3
						S48	225	142	1	56673	1333.1

less than 20 minutes. Since these facings can be different for alternate optimal solutions, we do not list the number of facings at the tables. Nevertheless, these results show the strength of the IP1/IP2 method clearly.

5.2. Experiments with *SyntheticData*

We compare the performance of MIP, LBBD1 and IP1/IP2 methods with 48 test instances in the *SyntheticData* in Table 7 (15 instances), and also construct Table 8 for large size instances (33 instances). As in the *RealisticData* dataset, the performance of MIP and LBBD1 are worse than the IP1/IP2 method. Results from Tables 7 and 8 show that our IP1/IP2 method outperforms MIP and LBBD1 and solves large size instances while achieving the rectangular arrangement of facings of all books at once.

Our results with two datasets show that the IP1/IP2 method is superior to MIP and LBBD1 and can solve practical and realistic instances of SSAP-RA optimally despite additional hard restrictions such as the different thicknesses of books and ensuring the sequence in both rows and columns at the same time, i.e., achieving rectangular arrangement. Therefore, we use the IP1/IP2 method to mimic the real-world application in the bookstore and to conduct the detailed analysis to determine profit improvement with different scenarios in the next section.

5.3. Experiments with *CaseData*

As mentioned before, we gather the data from the bookstore's bestseller cabinets and develop the *CaseData* dataset. The *CaseData* contains 557 different books with different facings in total. Considering the current placement, there are 634 facings at the cabinets. Using the exact location of books, the profit of books, and the effect parameters of rows, we calculated each cabinet's profit with our objective function. To ensure a fair comparison of profits between the bookstore's current allocation plan and the optimal plan that IP1/IP2 provided, we consider three different scenarios:

- **Scenario 1:** The IP1/IP2 model allocates the same books with the same facings (the lower bound of facings and the upper bound of facings are equal to the current facings) to the same area.
- **Scenario 2:** The IP1/IP2 model allocates the same books, the lower bound of facings are the current facings, the upper bound of facings are 4, and the total area is the same as the current allocation.
- **Scenario 3:** The IP1/IP2 model allocates the same books, the lower bound of facings are the current facings, the upper bound of facings are 4, and the total area is the same as the cabinet with the maximum currently used capacity.

Table 9

Profit comparison of current allocation and IP1/IP2 solutions for Scenario 1.

Cabinets	# of Rows	# of Cols	# of Books	Current profit	IP1/IP2 profit	IP1/IP2 Sol. time	Imp.%
RC1	8	47	134	9531	9929	74.88	4.18
RC2	8	52	133	10,778	11,305	58.70	4.89
RC3	8	51	136	9502	9945	37.31	4.66
RC4	8	49	154	9885	10,377	34.73	4.98
Average							4.68

Table 10

Profit comparison of current allocation and IP1/IP2 solutions for Scenario 2.

Cabinets	# of Rows	# of Cols	# of Books	Current profit	IP1/IP2 profit	IP1/IP2 Sol. time	Imp.%
RC1	8	47	134	9531	9929	71.55	4.18
RC2	8	52	133	10,778	11,305	41.58	4.89
RC3	8	51	136	9502	9945	34.55	4.66
RC4	8	49	154	9885	10,377	39.24	4.98
Average							4.68

The details of the scenarios and comparisons are provided in the next section.

5.3.1. Scenario 1 comparison results

In this scenario, we took books' upper bounds and lower bounds equal to the number of real facing units, and then the IP1/IP2 model allocated the same books with the same facing units to the same total area (i.e., the same number of columns and rows). We do not consider the actual cabinet size, which has 64 columns. Instead of using the actual size, we allowed empty spaces as in the current placement and assign books to the same total area. With this scenario, we investigated that whether IP1/IP2 improves profit by allocating books to better locations or not. In other words, we analyzed the impact of location-elastic demand on the profit. Table 9 shows the comparison results of the real-world allocations and IP1/IP2 solutions with scenario 1 parameters. Since we do not change the number of facings and the total area, the total improvement on the profit is only 4.68%, on average. Nevertheless, with the same books, same facings, and same area, the IP1/IP2 method improves books' allocations and increases profit by almost 5%.

5.3.2. Scenario 2 comparison results

In this scenario, we allocated the same books, used the actual facings as the lower bounds books' facings, took the upper bound of facings as 4 for all books, and used the same total area. With this scenario, we investigated that whether IP1/IP2 improves the profit by allocating books to better locations with possible different facings. Since we use the same area with the current arrangement, the results are same with the first scenario, which is 4.68%, on average. The comparison results are presented in Table 10.

5.3.3. Scenario 3 comparison results

In this scenario, we investigated the profit improvement of IP1/IP2 if we use the total area that the bookstore is current using at cabinets. Namely, we analyzed the impact of space-elasticity on the profit. Thus, rather than using the actual area (64 columns), we took the cabinets' size as 8x52 slots, which is applied by the bookstore in real allocations. We allocated the same books, used the actual facings as the lower bounds books' facings, and took the upper bound of facings as 4 for all books. Since we allow IP1/IP2 to use the maximum capacity at the real allocation, the model can assign more facings of books, and the profit is increased, as expected. Table 11 shows the comparison results of the current allocation and IP1/IP2 with scenario 3 parameters. We see a signifi-

cant gain by the model, 16.56% improvement in profits, on average. IP1/IP2 can solve this real-world SSAP-RA in minutes and provides significant profit increase.

There is another scenario in which we can calculate an upper bound on the profit potential in implementing our models. If we use the current length of a row (a bookshelf length in the bookstore which is 64cm) in our slot calculation with the thinnest book (1cm), then we calculate the number of columns as 64. If we use 8x64 available shelf space with 4 cabinets and allocate the same books with the same facing assumptions in scenario 2 or scenario 3, the average profit improvement becomes 59.62%. This could be possible if the bookstore was utilizing all its available shelf space. However, to provide a more realistic improvement estimate according to current practices of the bookstore, we only present this number as a potential and do not report as one of the scenarios.

One reason why sometimes the bookstore cannot utilize the entire bookshelf space is that the managers are overwhelmed with the frequency of developing an allocation plan for bookshelves. With the help of IP1/IP2, we think and hope that they will come up with plans quicker and revise them more often and hence increase their profits.

6. Conclusion

We aim to solve SSAPs-RA by achieving allocation of facings of products in two dimensions in rectangular arrangement, and we investigate efficient solution methods for solving them.

We first construct a mixed-integer mathematical programming model (MIP) for SSAP-RA. Although MIP can solve small size instances of the problem, it cannot even find feasible solutions for the real-world size problems. When we investigate the performance of the MIP model, we apprehend that the consecutiveness constraints (for rows and columns) make the problem harder. Thus, we decompose the problem by using LBBD and develop the LBBD1 algorithm. Whilst LBBD1 performs better than MIP, it also falls short of solving larger instances. Then we realize the relationship between row and column consecutiveness constraints and decompose the problem by introducing a 2-stage iterative approach, IP1/IP2. IP1 provides the sequence of facings in rows, and IP2 takes IP1's solution and achieves the sequence of facings in columns. We test our models' performance with two different datasets: the *RealisticData* dataset, includes 48 test instances, generated from a real-world application of a local bookstore and the *Synthetic* dataset, includes 48 test instances, generated from the benchmarks in the

Table 11
Profit comparison of current allocation and IP1/IP2 solutions for Scenario 3.

Cabinets	# of Rows	# of Cols	# of Books	Current profit	IP1/IP2 profit	IP1/IP2 Sol. time	Imp.%
RC1	8	52	134	9531	12,369	31.56	27.78
RC2	8	52	133	10,778	11,305	44.90	4.89
RC3	8	52	136	9502	10,686	68.75	12.46
RC4	8	52	154	9885	11,973	70.15	21.12
Average							16.56

literature. IP1/IP2 performs well and the best in all these tests, and it has the potential to improve profits within 4.68% - 16.56% for the bookstore, which is obtained with the *CaseData* dataset that includes real-world allocations of products with 4 test instances.

Although the IP1/IP2 method can find the optimal solutions of real-world SSAPs-RA, it has limitations. The main reason why IP1/IP2 works is that the objective function is not affected by the column location of the facing units. If the problem also includes different profit values for column locations, a different approach may be needed. This would also change the definition of SSAP-RA and calls for future research. Another area for further research may be the investigation of heuristic methods for solving even larger instances. In addition, different two-dimensional assignment and allocation problems such as allocation of categories of products (product families) in a rectangular arrangement at grocery stores or advertising allocation in webpages can be investigated, and the 2-stage algorithm can be extended for solving problems similar to SSAPs-RA. These could be particularly interesting for companies who practice omnichannel retailing to satisfy users' expectations, such as a seamless shopping experience, ensure customer loyalty, maintain competitiveness, and acquire new customers. Omnichannel retailing strategy has become mandatory to compete in retailing, which increases the importance of effective shelf space management. To implement omnichannel retailing, it is necessary to allow customers to check the availability of the products they want through online channels, improve retail stores to provide multiple delivery points, and provide synchronization between online channels and physical stores. The 2-stage algorithm can be effectively used to produce fast and optimal allocations, and hence retail shelves can be more easily adapted to rapidly changing online channels as well as synchronization can be achieved between online channels and physical stores.

Acknowledgment

This study was supported by TUBITAK (The Scientific and Technological Research Council of Turkey), under Grant no. 217M920. We thank Beha Bookstore for providing us their shelf allocation problem definition and data for our paper.

Appendix A. The proofs for Benders cuts in LBBD1

Two properties of a *valid* Benders cuts (Chu & Xia, 2004) are:

Property 1. If the current solution of the master problem is not globally feasible, the cut must eliminate at least this solution from the solution space.

Property 2. The cut cannot eliminate any globally feasible solutions but the current solution.

Property 1 guarantees a finite convergence if the variables have a finite definition space, and Property 2 guarantees optimality because none of the globally feasible solutions is eliminated rather than the current solution. We show these properties are valid for our Benders optimality cuts and feasibility cuts.

Proposition 1. The Benders feasibility cut (26) is valid.

Proof. We define E^h as the MIP1 solution at iteration h and let set $T(E^h)$ be all the t_{dijk} decision variables taking value 1 at solution E^h . Then, for any future solution F with set $T(F)$, we have two cases:

- $T(E^h) \cap T(F) = T(E^h)$ or
- $T(E^h) \cap T(F) \neq T(E^h)$

- If $T(E^h) \cap T(F) = T(E^h)$ then for solution F , (26) becomes

$$\sum_{S_{dijk}^h} \underbrace{(1 - t_{dijk})}_0 \not\geq 1$$

therefore solution F (correctly) does not satisfy the equation and it is removed.

- If $T(E^h) \cap T(F) \neq T(E^h)$, then (26) becomes

$$\underbrace{\sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \setminus T(F)} (1 - t_{dijk})}_{\geq 1} + \underbrace{\sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \cap T(F)} (1 - t_{dijk})}_0 \geq 1$$

hence solution F satisfies the cut and it is (correctly) not removed. $\square$

Proposition 2. The Benders optimality cut (27) is valid.

Proof. Similar to the proof of the feasibility cut (Proposition 1), we have the same two cases:

- $T(E^h) \cap T(F) = T(E^h)$ or
- $T(E^h) \cap T(F) \neq T(E^h)$

- For such solution F we have

$$Z_{MIP1} \leq Z_{SMIP1} * (1 - \sum_{S_{dijk}^h} \underbrace{(1 - t_{dijk})}_0) + M * \sum_{S_{dijk}^h} \underbrace{(1 - t_{dijk})}_0$$

$$\Rightarrow Z_{MIP1} \leq Z_{SMIP1},$$

i.e., subproblem correctly provides an upper bound on the master problem.

- In this case, we have

$$Z_{MIP1} \leq Z_{SMIP1} * [1 - \underbrace{\sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \setminus T(F)} (1 - t_{dijk})}_{\geq 1} - \underbrace{\sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \cap T(F)} (1 - t_{dijk})}_0 + M * \underbrace{\sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \cap T(F)} (1 - t_{dijk})}_0 + M * \underbrace{\sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \setminus T(F)} (1 - t_{dijk})}_{\geq 1}]$$

$$\Rightarrow Z_{MIP1} \leq Z_{SMIP1} + M_{new}$$

where

$$M_{new} = 1 + M_{temp} * (M - 1)$$

and

$$M_{temp} = \sum_{S_{dijk}^h: t_{dijk} \in T(E^h) \setminus T(F)} (1 - t_{dijk}) \geq 1$$

so Z_{MIP1} is (correctly) not constrained. $\square$

Appendix B. IP1/IP2 optimality proof

We prove the optimality of IP1/IP2 model in Proposition 3, but before we need a Lemma.

Lemma 1. *If IP1 is feasible, then IP2 is feasible. Hence optimal solution of IP2 exists and it has the same optimal objective value.*

Proof. If IP1 is feasible, then since it is bounded its optimum exists, and hence we can fix the values of x_{dijk} , n_{dik} , m_{dk} , w_d , h_d and y_{dik} variables for IP2. Then IP2 becomes an optimization with the same objective function, all the IP1 constraints, column consecutiveness constraint (23), additional constraints to fix x_{dijk} , n_{dik} , m_{dk} , w_d , h_d and y_{dik} variables, and decision variables of t_{dijk} , p_{djk} , and z_{djk} .

For IP2, constraints (7), (11–14), (16–17), (20–22), and domain of variables are satisfied by IP1 solution. The remaining constraints (8–10), (15), (18–19), and (23) are satisfied by choosing appropriate values of only t_{dijk} , p_{djk} , and z_{djk} variables. This is possible since IP1 makes sure each product is placed only in one cabinet in consecutive h_d number of rows with exactly w_d number of units in each row and hence IP2 simply changes the location of each unit within a row to achieve rectangular display for each product in a cabinet. And n_{dik} values are the same for both models, i.e., number of units of a particular product in a row of a cabinet stays the same. Therefore, both models have the same objective value in their optimal solutions. $\square$

Next, we show that IP2 achieves the same optimal value as MIP. Therefore, we can use IP1/IP2 method to solve our SSAP-RA efficiently and can solve larger instances. Let Υ_α be the set of decision variables in models for model $\alpha \in \{MIP, IP1, IP2\}$.

Let $Z_\beta(\Upsilon_\alpha)$ be the value of the objective function value of model $\beta \in \{MIP, IP1, IP2\}$ with optimal solution Υ_α for $\alpha \in \{MIP, IP1, IP2\}$. We will show that optimal value of MIP and IP2 are the same, i.e.,

$$Z_{MIP}(\Upsilon_{MIP}) = Z_{IP2}(\Upsilon_{IP2}) = Z_{MIP}(\Upsilon_{IP2})$$

Proposition 3. $Z_{MIP}(\Upsilon_{MIP}) = Z_{IP2}(\Upsilon_{IP2}) = Z_{MIP}(\Upsilon_{IP2})$.

Proof.

$Z_{IP2}(\Upsilon_{IP2})$	$\leq$	$Z_{MIP}(\Upsilon_{IP2})$	(MIP is a relaxation of IP2)
$Z_{MIP}(\Upsilon_{IP2})$	$\leq$	$Z_{MIP}(\Upsilon_{MIP})$	(Optimality of Υ_{MIP})
$Z_{MIP}(\Upsilon_{MIP})$	$\leq$	$Z_{IP1}(\Upsilon_{MIP})$	(IP1 is a relaxation of MIP)
$Z_{IP1}(\Upsilon_{MIP})$	$\leq$	$Z_{IP1}(\Upsilon_{IP1})$	(Optimality of Υ_{IP1})
$Z_{IP1}(\Upsilon_{IP1})$	$=$	$Z_{IP2}(\Upsilon_{IP2})$	(by Lemma 1).

Then all inequalities must be equalities, hence $Z_{MIP}(\Upsilon_{MIP}) = Z_{IP2}(\Upsilon_{IP2})$. Then, $Z_{MIP}(\Upsilon_{MIP}) = Z_{MIP}(\Upsilon_{IP2})$ since IP2 and MIP have the same objective function and the same value for n_{dik} variables at their optimal solutions, and Υ_{IP2} is feasible for MIP. Thus, it suffices to solve IP2 and use Υ_{IP2} for MIP. $\square$

References

- Anderson, E. E., & Amato, H. N. (1974). A mathematical model for simultaneously determining the optimal brand-collection and display-area allocation. *Operations Research*, 22(1), 13–21. <https://doi.org/10.1287/opre.22.1.13>.
- Bai, R., Van Woensel, T., Kendall, G., & Burke, E. K. (2013). A new model and a hyper-heuristic approach for two-dimensional shelf space allocation. *4or*, 11(1), 31–55.
- Barzanji, R., Naderi, B., & Begen, M. A. (2020). Decomposition algorithms for the integrated process planning and scheduling problem. *Omega*, 93, 102025. <https://doi.org/10.1016/j.omega.2019.01.003>.
- Bianchi-Aguiar, T. (2015). *The retail shelf space allocation problem: New optimization methods applied to a supermarket chain*. Porto University Ph.D. thesis.
- Bianchi-Aguiar, T., Hübner, A., Carravilla, M. A., & Oliveira, J. F. (2021). Retail shelf space planning problems: A comprehensive review and classification framework. *European Journal of Operational Research*, 289, 1–16. <https://doi.org/10.1016/j.ejor.2020.06.018>.
- Bianchi-Aguiar, T., Silva, E., Guimarães, L., Carravilla, M. A., & Oliveira, J. F. (2018). Allocating products on shelves under merchandising rules: Multi-level product families with display directions. *Omega*, 76, 47–62. <https://doi.org/10.1016/j.omega.2017.04.002>.

- Bonami, P., & Lee, J. (2007). BONMIN users' manual. <http://www.coin-or.org/Bonmin>. Accessed March 16, 2021.
- Borin, N., Farris, P. W., & Freeland, J. R. (1994). A model for determining retail product category assortment and shelf space allocation. *Decision Sciences*, 25(3), 359–384. <https://doi.org/10.1111/j.1540-5915.1994.tb00809.x>.
- Brown, M. G., & Lee, J.-Y. (1996). Allocation of shelf space: A case study of refrigerated juice products in grocery stores. *Agribusiness: An International Journal*, 12(2), 113–121.
- Caglar Gencosman, B., & Begen, M. (2021). Data for: Exact optimization and decomposition approaches for shelf space allocation. *Mendeley Data*, V2. <https://doi.org/10.17632/zy6bm4nr9w.2>.
- Caglar Gencosman, B., Begen, M. A., Ozmutlu, H. C., & Ozturk Yilmaz, I. (2016). Scheduling methods for efficient stamping operations at an automotive company. *Production and Operations Management*, 25(11), 1902–1918. <https://doi.org/10.1111/poms.12584>.
- Camargo, V. C. B., Toledo, F. M. B., & Almada-Lobo, B. (2014). Hops-hamming-oriented partition search for production planning in the spinning industry. *European Journal of Operational Research*, 234(1), 266–277. <https://doi.org/10.1016/j.ejor.2013.10.017>.
- Cambazard, H., Hladik, P. E., Déplanche, A. M., Jussien, N., & Trinquet, Y. (2004). Decomposition and learning for a hard real time task allocation problem. In *Principles and practice of constraint programming-CP 2004* (pp. 153–167). Springer, Berlin Heidelberg.
- Chandon, P., Hutchinson, J. W., Bradlow, E. T., & Young, S. H. (2009). Does in-store marketing work? effects of the number and position of shelf facings on brand attention and evaluation at the point of purchase. *Journal of Marketing*, 73(6), 1–17. <https://doi.org/10.1509/jmkg.73.6.1>.
- Chen, M.-C., & Lin, C.-P. (2007). A data mining approach to product assortment and shelf space allocation. *Expert Systems with Applications*, 32(4), 976–986. <https://doi.org/10.1016/j.eswa.2006.02.001>.
- Chen, Y.-L., Chen, J.-M., & Tung, C.-W. (2006). A data mining approach for retail knowledge discovery with consideration of the effect of shelf-space adjacency on sales. *Decision Support Systems*, 42(3), 1503–1520. <https://doi.org/10.1016/j.dss.2005.12.004>.
- Chu, Y., & Xia, Q. (2004). Generating benders cuts for a general class of integer programming problems. In *Integration of AI and OR techniques in constraint programming for combinatorial optimization problems* (pp. 127–141). Springer.
- Coban, E., & Hooker, J. N. (2010). Single-facility scheduling over long time horizons by logic-based benders decomposition. In *Integration of AI and OR techniques in constraint programming for combinatorial optimization problems* (pp. 87–91). Springer, Berlin Heidelberg.
- Corstjens, M., & Doyle, P. (1981). A model for optimizing retail space allocations. *Management Science*, 27(7), 822–833. <https://doi.org/10.1287/mnsc.27.7.822>.
- Coskun, M. E. (2012). *Shelf space allocation: A critical review and a model with price changes and adjustable shelf heights*. McMaster University Ph.D. thesis.
- Côté, J.-F., Dell'Amico, M., & Iori, M. (2014). Combinatorial benders' cuts for the strip packing problem. *Operations Research*, 62(3), 643–661. <https://doi.org/10.1287/opre.2013.1248>.
- Cox, K. K. (1970). The effect of shelf space upon sales of branded products. *Journal of Marketing Research*, 7(1), 55–58.
- Curhan, R. C. (1972). The relationship between shelf space and unit sales in supermarkets. *Journal of Marketing Research*, 406–412. <https://doi.org/10.1177/00224377200900408>.
- Delorme, M., Iori, M., & Martello, S. (2017). Logic based benders' decomposition for orthogonal stock cutting problems. *Computers & Operations Research*, 78, 290–298. <https://doi.org/10.1016/j.cor.2016.09.009>.
- Dreze, X., Hoch, S. J., & Purk, M. E. (1994). Shelf management and space elasticity. *Journal of Retailing*, 70(4), 301–326. [https://doi.org/10.1016/0022-4359\(94\)90002-7](https://doi.org/10.1016/0022-4359(94)90002-7).
- Fazel-Zarandi, M. M., & Beck, J. C. (2009). Solving a location-allocation problem with logic-based benders' decomposition. In *Principles and practice of constraint programming-CP 2009* (pp. 344–351). Springer, Berlin Heidelberg.
- Frontoni, E., Marinelli, F., Rosetti, R., & Zingaretti, P. (2017). Shelf space re-allocation for out of stock reduction. *Computers & Industrial Engineering*, 106, 32–40. <https://doi.org/10.1016/j.cie.2017.01.021>.
- Geismar, H. N., Dawande, M., Murthi, B., & Sriskandarajah, C. (2015). Maximizing revenue through two-dimensional shelf-space allocation. *Production and Operations Management*, 24(7), 1148–1163. <https://doi.org/10.1111/poms.12316>.
- Hansen, J. M., Raut, S., & Swami, S. (2010). Retail shelf allocation: A comparative analysis of heuristic and meta-heuristic approaches. *Journal of Retailing*, 86(1), 94–105. <https://doi.org/10.1016/j.jretai.2010.01.004>.
- Hansen, P., & Heinsbroek, H. (1979). Product selection and space allocation in supermarkets. *European Journal of Operational Research*, 3(6), 474–484. [https://doi.org/10.1016/0377-2217\(79\)90030-4](https://doi.org/10.1016/0377-2217(79)90030-4).
- Heinz, S., & Beck, J. C. (2012). Reconsidering mixed integer programming and MIP-based hybrids for scheduling. In *Integration of AI and OR techniques in constraint programming for combinatorial optimization problems* (pp. 211–227). Springer, Berlin Heidelberg.
- Hooker, J. N. (2005). Planning and scheduling to minimize tardiness. In *Principles and practice of constraint programming-CP 2005* (pp. 314–327). Springer, Berlin Heidelberg.
- Hooker, J. N. (2007). Planning and scheduling by logic-based benders decomposition. *Operations Research*, 55(3), 588–602. <https://doi.org/10.1287/opre.1060.0371>.

- Hooker, J. N. (2019). Logic-based benders decomposition for large-scale optimization. In *Large scale optimization in supply chains and smart manufacturing* (pp. 1–26). Springer.
- Hooker, J. N., & Ottosson, G. (2003). Logic-based benders decomposition. *Mathematical Programming*, 96(1), 33–60. <https://doi.org/10.1007/s10107-003-0375-9>.
- Hübner, A. (2017). A decision support system for retail assortment planning. *International Journal of Retail & Distribution Management*, 45(7/8), 808–825. <https://doi.org/10.1108/IJRDM-09-2016-0166>.
- Hübner, A., Schäfer, F., & Schaal, K. N. (2021). Maximizing profit via assortment and shelf-space optimization for two-dimensional shelves. *Production and Operations Management*, 29(3), 547–570. <https://doi.org/10.1111/poms.13111>.
- Hübner, A. H., & Kuhn, H. (2012). Retail category management: State-of-the-art review of quantitative research and software applications in assortment and shelf space management. *Omega*, 40(2), 199–209. <https://doi.org/10.1016/j.omega.2011.05.008>.
- Hwang, H., Choi, B., & Lee, G. (2009). A genetic algorithm approach to an integrated problem of shelf space design and item allocation. *Computers & Industrial Engineering*, 56(3), 809–820.
- Kök, A. G., Fisher, M. L., & Vaidyanathan, R. (2015). Assortment planning: Review of literature and industry practice. In *Retail supply chain management* (pp. 175–236). Springer.
- Kotzan, J. A., & Evanson, R. V. (1969). Responsiveness of drug store sales to shelf space allocations. *Journal of Marketing Research*, 6(4), 465–469.
- Martello, S., & Toth, P. (1990). An exact algorithm for large unbounded knapsack problems. *Operations research letters*, 9(1), 15–20. [https://doi.org/10.1016/0167-6377\(90\)90035-4](https://doi.org/10.1016/0167-6377(90)90035-4).
- Martin, M., Hokama, P. H., Morabito, R., & Munari, P. (2020). The constrained two-dimensional guillotine cutting problem with defects: An ILP formulation, a benders decomposition and a CP-based algorithm. *International Journal of Production Research*, 58(9), 2712–2729. <https://doi.org/10.1080/00207543.2019.1630773>.
- Murray, C. C., Talukdar, D., & Gosavi, A. (2010). Joint optimization of product price, display orientation and shelf-space allocation in retail category management. *Journal of Retailing*, 86(2), 125–136. <https://doi.org/10.1016/j.jretai.2010.02.008>.
- Naderi, B., Roshanaei, V., Begen, M. A., Aleman, D. M., & Urbach, D. R. (2021). Increased surgical capacity without additional resources: Generalized operating room planning and scheduling. *Production and Operations Management*. <https://doi.org/10.1111/poms.13397>.
- Nafari, M., & Shahrabi, J. (2010). A temporal data mining approach for shelf-space allocation with consideration of product price. *Expert Systems with Applications*, 37(6), 4066–4072. <https://doi.org/10.1016/j.eswa.2009.11.045>.
- Özcan, T. (2010). Retail shelf space management: Literature review and design of a decision support tool. *Istanbul Management Journal*, 21(67), 84–103.
- Özcan, T. (2011). *An analytic approach based on data mining to shelf space management in retail industry*. Istanbul University Ph.D. thesis.
- Rabbani, M., Salmanzadeh-Meydani, N., Farshbaf-Geranmayeh, A., & Fadakar-Gabalou, V. (2018). Profit maximizing through 3d shelf space allocation of 2d display orientation items with variable heights of the shelves. *Opsearch*, 55(2), 337–360. <https://doi.org/10.1007/s12597-018-0335-z>.
- Russell, R. A., & Urban, T. L. (2010). The location and allocation of products and product families on retail shelves. *Annals of Operations Research*, 179(1), 131–147. <https://doi.org/10.1007/s10479-008-0450-y>.
- Tran, T. T., & Beck, J. C. (2012). Logic-based benders decomposition for alternative resource scheduling with sequence dependent setups. In *In proceedings of the 20th European conference on artificial intelligence (ECAI), Montpellier, France*.
- Urban, G. L. (1969). A mathematical modeling approach to product line decisions. *Journal of Marketing Research*, 6(1), 40–47.
- Van Nierop, E., Fok, D., & Franses, P. H. (2008). Interaction between shelf layout and marketing effectiveness and its impact on optimizing shelf arrangements. *Marketing Science*, 27(6), 1065–1082. <https://doi.org/10.1287/mksc.1080.0365>.
- Xiaolu, L., Baocun, B., Yingwu, C., & Feng, Y. (2014). Multi satellites scheduling algorithm based on task merging mechanism. *Applied Mathematics and Computation*, 230, 687–700. <https://doi.org/10.1016/j.amc.2013.12.109>.
- Yang, M.-H., & Chen, W.-C. (1999). A study on shelf space allocation and management. *International Journal of Production Economics*, 60, 309–317. [https://doi.org/10.1016/S0925-5273\(98\)00134-0](https://doi.org/10.1016/S0925-5273(98)00134-0).
- Zhao, J., Zhou, Y.-W., & Wahab, M. (2016). Joint optimization models for shelf display and inventory control considering the impact of spatial relationship on demand. *European Journal of Operational Research*, 255(3), 797–808. <https://doi.org/10.1016/j.ejor.2016.05.025>.
- Zufryden, F. S. (1986). A dynamic programming approach for product selection and supermarket shelf-space allocation. *Journal of the Operational Research Society*, 37(4), 413–422.